

Dew Computing & Serverless Computing

Complete Beginner-Friendly Revision Notes

What You Will Learn

Part 1 — Dew Computing (Deep Concept)

What it is, why it exists, how it works offline,
how it fits with Cloud, Fog and Edge computing,
real-life examples, the naming story, and trade-offs

Part 2 — Serverless Computing (Deep Concept)

What serverless means, what a function is,
how FaaS (Function-as-a-Service) works,
cold start problem, stateless design, and limitations

Part 3 — MCQ Solution Set (10 Questions)

Every question explained step by step from scratch
with real-life analogies before each exam answer

Story-Driven • Zero Jargon • Exam-Ready

Suitable for complete beginners

Contents

1	Dew Computing — The Ground Layer of the Cloud Hierarchy	2
1.1	Why Does Dew Computing Even Exist?	2
1.2	What is Dew Computing? — The Core Idea	2
1.3	How Dew Computing Works — Step by Step	3
1.4	Real-Life Examples That Make It Click	3
1.4.1	Google Docs Offline Mode	3
1.4.2	Offline Mobile Games	4
1.4.3	Village School with Unreliable Internet	4
1.4.4	Health Worker in Remote Areas	4
1.5	Dew Computing vs Other Paradigms	5
1.6	The Trade-Off in Dew Computing	5
2	Serverless Computing — Run Code Without Managing Servers	7
2.1	The Problem That Serverless Solves	7
2.2	What is Serverless Computing?	7
2.3	What is a Serverless Function?	8
2.4	How a Serverless Function Works — Step by Step	8
2.5	FaaS — Function as a Service	9
2.6	Stateless Design — Why It Matters	9
2.7	The Cold Start Problem	10
2.8	Limitations of Serverless and FaaS	10
3	MCQ Solutions — Dew Computing & Serverless	12
3.1	Q1 Dew Computing vs Edge Computing — The Key Difference . .	12
3.2	Q2 Best Use Case for Dew Computing	13
3.3	Q3 Who Resolves Data Conflicts After Offline Work?	13
3.4	Q4 The Core Trade-Off in Dew Computing	14
3.5	Q5 The Cold Start Problem in Serverless Computing	15
3.6	Q6 Why Serverless Functions Must Be Stateless	16
3.7	Q7 Why FaaS is Not Suitable for Long-Running Tasks	17
3.8	Q8 Why Local File Storage is Unreliable in FaaS	18
3.9	Q9 AWS Lambda — Maximum Execution Time	19
3.10	Q10 Amazon Redshift — What Is It and Who Provides It?	19
4	Master Quick-Reference Sheet	21
4.1	Dew Computing — Key Facts	21
4.2	Serverless Computing — Key Facts	22
4.3	The Computing Hierarchy — All Layers Together	22
4.4	Five Golden Rules to Remember	23

1 Dew Computing — The Ground Layer of the Cloud Hierarchy

1.1 Why Does Dew Computing Even Exist?

Before learning what dew computing is, it helps to understand the problem it was created to solve.

Almost every modern computing system depends on the internet. Cloud computing assumes you always have a fast, stable network connection. Edge and fog computing assume you have at least some network nearby. But what about billions of people in the world who live in areas with poor, expensive, or completely absent internet? What about the engineer working underground where there is no signal, or the student in a village where the power goes out along with the internet?

The Fundamental Problem:

Most computing models treat the internet as a **given**. But for a huge portion of the world and for many everyday situations, the internet is **not reliable**. When connectivity disappears, cloud-dependent apps stop working entirely. Work is lost. People are stuck.

Dew computing was proposed to solve exactly this. The idea is simple but powerful: make your **own device capable enough to work completely on its own**, and only use the cloud when a connection is available.

1.2 What is Dew Computing? — The Core Idea

Dew Computing is a computing model where the primary work is done **locally on your device** (laptop, phone, desktop), and the cloud is used only for backup, synchronization, and storage when connectivity is available.

One-Line Definition:

Dew computing = **Work locally first. Sync with the cloud later.**

Your device is fully capable of doing its job with or without the internet. The cloud is a bonus, not a requirement.

The name comes from a beautiful natural metaphor. In nature, water exists in three states based on height:

Layer	Natural Form	Computing Equivalent
Highest	Cloud (in the sky)	Cloud Computing — remote data centers
Middle	Fog (near the ground)	Fog/Edge — nearby local servers
Lowest	Dew (on the ground)	Dew Computing — your own device

Just like *dew* is water that has come all the way down to the ground level and is available right where you are standing, *dew computing* brings computation all the way down to the device in your hand.

1.3 How Dew Computing Works — Step by Step

Let us walk through exactly what happens in a dew computing setup:

Step 1 — You Do Your Work:

You open an application on your device and start working. Writing a document, editing a spreadsheet, filling a form, recording data. Everything happens **right on your device**. No internet request is made. The app does not freeze. Work continues smoothly.

Step 2 — Data is Saved Locally:

As you work, all your data is saved to your device's local storage — your phone's internal memory, your laptop's hard drive, or a local database. This is your primary copy. It is always with you.

Step 3 — Internet Becomes Available:

At some point — maybe when you reach a WiFi zone, or when your mobile data returns, or when the village internet comes back — the application detects connectivity.

Step 4 — Automatic Synchronization:

The application automatically syncs your locally saved work to the cloud server. It checks what changed, what is new, and what needs to be updated. This happens quietly in the background. You do not have to do anything manually.

The result is a system that is **completely transparent to the user**. Whether you are online or offline, you just keep working. The system handles everything else.

1.4 Real-Life Examples That Make It Click

1.4.1 Google Docs Offline Mode

If you have ever used Google Docs with offline mode turned on, you have experienced dew computing. You write a document with no internet — the words are saved on your device. The moment you reconnect, Google syncs everything to the cloud automatically. The document is now accessible from all your other devices too.

1.4.2 Offline Mobile Games

You play a game on your phone during a flight with airplane mode on. Every coin, every level, every score is saved locally on your phone. When you land and reconnect, the game syncs your progress to the server. Your rank on the global leaderboard updates. This is dew computing in everyday entertainment.

1.4.3 Village School with Unreliable Internet

This is perhaps the most important real-world use case. Imagine a school in a rural area where internet is available only for two hours each day, and power cuts are frequent.

Without Dew Computing:

The school uses an online learning platform. When the internet goes down — which happens three times a day — students cannot access lessons, teachers cannot record attendance, and no progress is saved. The school day is constantly interrupted.

With Dew Computing:

The learning app stores all lessons, assignments, and student data locally on each device. Teachers take attendance on their tablets offline. Students read lessons, submit assignments, and take quizzes — all without internet. When the two-hour internet window opens, everything syncs to the central cloud server. The school runs perfectly regardless of connectivity.

1.4.4 Health Worker in Remote Areas

A health worker visits patients in villages far from the city. They record patient data — symptoms, prescriptions, diagnoses — on a tablet with no network coverage. All data is stored locally. When they return to the district hospital and connect to WiFi, all patient records sync to the central hospital database automatically. No data is lost, no paper forms are needed.

1.5 Dew Computing vs Other Paradigms

Feature	Cloud	Edge	Fog	Dew
Where it runs	Remote server	Near device	Middle layer	Your own device
Internet needed	Always	Mostly	Partially	Not required
Works offline?	No	Rarely	Sometimes	Yes, fully
Dependency	Very high	Medium	Low	Very low
Who stores data	Server	Nearby node	Fog node	Your device
Best use case	Big computation	Real-time IoT	Smart cities	Offline-first apps

1.6 The Trade-Off in Dew Computing

Nothing in technology is free. Dew computing gives you a major advantage (offline independence), but it introduces a real challenge that must be carefully managed.

What You Gain:

- ✓ Complete independence from internet connectivity
- ✓ No interruption to work when network is unavailable
- ✓ Faster response time (no need to wait for server)
- ✓ Works in remote, rural, or travel environments
- ✓ Reduced data costs (less continuous network usage)

What You Give Up — The Data Consistency Problem:

- ✗ If two people edit the same file offline simultaneously, both create different versions
- ✗ When both sync to the cloud, there is a **conflict** — two different “correct” versions of the same data
- ✗ The system must decide which version wins, or merge them, or ask the user
- ✗ Getting this conflict resolution right is technically very complex
- ✗ If handled badly, data can be lost or corrupted

The Conflict Problem — Made Very Simple:

You and your classmate both download a group project file at 10am. You edit it offline all day. Your classmate also edits it offline all day. At 6pm, both of you connect to the internet and try to sync.

Now the cloud has a problem: your version says the project has 10 pages. Your classmate's version says it has 15 pages. Both are different from the version that was there at 10am. Which one should the cloud keep? What if you both added different chapters?

This is the **data consistency trade-off** of dew computing. The freedom of offline work creates the challenge of version conflicts when syncing.

The cloud layer handles this conflict resolution — it compares timestamps, content differences, and sometimes asks the user to choose which version to keep.

2 Serverless Computing — Run Code Without Managing Servers

2.1 The Problem That Serverless Solves

Imagine you want to build a small feature for your app: when a user uploads a profile picture, automatically resize it to a smaller version and save it. That is a very simple job — maybe 10 lines of code.

But to run that 10-line code the traditional way, you would need to:

1. Rent a server (a cloud computer) and pay for it 24 hours a day, 7 days a week
2. Install an operating system on the server
3. Install the right programming language and libraries
4. Configure security, firewalls, and network settings
5. Deploy your code to the server
6. Monitor the server in case it crashes
7. Scale it manually if many users upload at the same time

The Core Problem:

You wrote 10 lines of code, but now you are managing an entire server infrastructure. The server runs all day even when nobody is uploading photos. You are paying for a computer to sit idle most of the time. You are doing work that has nothing to do with your actual task.

Serverless computing eliminates all of that.

2.2 What is Serverless Computing?

Serverless computing is a model where you write your code, hand it to a cloud provider, and the provider takes care of **everything else** — the server, the operating system, the scaling, the monitoring, and the billing. You pay only for the exact moments when your code is actually running.

One-Line Definition:

Serverless = **You write the code. The cloud runs it. You pay only when it runs.**

The word “serverless” is a little misleading — servers definitely exist. But from your perspective as a developer, you never see them, never configure them, never

manage them. They are invisible. That is what makes it “serverless” for you.

2.3 What is a Serverless Function?

In serverless computing, the unit of work is a **serverless function** — a small, self-contained piece of code that performs one specific task. It has a beginning, it does its job, and then it ends. It does not keep running after finishing.

Restaurant Analogy — The Best Way to Understand This:

Imagine you want a meal. You have two options:

Option A — Traditional Server Approach:

You hire a private chef. The chef comes to your home and waits in your kitchen all day, every day, just in case you want food. Even when you are at college or sleeping, the chef is there. You pay the chef’s full salary regardless.

Option B — Serverless Approach:

You call a restaurant when you are hungry. They prepare exactly what you ordered. You pay only for that meal. When you are not hungry, you pay nothing. The restaurant handles the kitchen, the staff, and the equipment.

Serverless functions are Option B. You only pay when the code is actually running.

2.4 How a Serverless Function Works — Step by Step

Step 1 — Write the Function:

You write a small piece of code that does one specific job. For example: “Take this image file, make it 200px by 200px, save it.” That is your function.

Step 2 — Upload to a Provider:

You upload this function to a serverless platform like AWS Lambda, Google Cloud Functions, or Azure Functions. The provider stores your code and waits.

Step 3 — Something Triggers It:

A trigger is an event that tells the function to run. Common triggers include: a user uploading a file, a user submitting a form, a timer going off at midnight, an API request arriving, or a message appearing in a queue.

Step 4 — Function Runs:

The cloud provider instantly starts a container, loads your function into it, runs it, and your function does its job.

Step 5 — Function Stops:

Once the job is done, the function container stops. Nothing continues running. No server is idle. No cost is incurred until the next trigger.

2.5 FaaS — Function as a Service

FaaS (Function as a Service) is the specific type of serverless computing where individual functions are the deployable unit. AWS Lambda is the most famous example. Google Cloud Functions and Azure Functions are others.

The main characteristics of FaaS are:

Event-driven: Functions do not run on their own. Something must trigger them — a user action, a scheduled time, a file upload, or an incoming message.

Stateless: Each function run is completely independent. The function does not remember anything from previous runs. It starts fresh every single time.

Short-lived: Functions are not meant to run for hours. They are designed for tasks that finish in seconds or a few minutes at most.

Auto-scaling: If 1000 users all upload photos at the same moment, the cloud automatically runs 1000 copies of your function in parallel. You do not configure anything — it just happens.

2.6 Stateless Design — Why It Matters

Stateless means the function has no memory of past events. Every time it runs, it is as if it is running for the very first time.

Delivery Worker Analogy:

Imagine a delivery worker whose only job is to deliver parcels. Each morning they receive a parcel, deliver it, and go home. The next morning they receive a new parcel and deliver it. They do not need to remember yesterday's delivery to do today's delivery. Each job is self-contained and independent.

That is stateless behavior. The function receives its input, does its work, produces an output, and ends — with no dependency on what happened before or what will happen after.

Why Stateless is a Good Thing for Serverless:

- ✓ Any instance of the function can handle any request — no need to route specific users to specific machines
- ✓ The cloud can run thousands of copies simultaneously without coordination
- ✓ If one copy crashes, another copy picks up the next request with no issue
- ✓ No data leaks between different users' requests

If a function needs to remember something (like a user's session), it stores that information in an external database or cache and reads it at the start of each run. The function itself remains stateless.

2.7 The Cold Start Problem

This is one of the most important limitations of serverless computing, and it comes up in almost every exam.

What is a Cold Start?

A serverless function does not run continuously. When not in use, it goes to sleep — the container is shut down completely to save resources. When a trigger arrives, the system must **wake it up**: start a container, load the runtime environment, load your code, and then run it. This setup time is called a **cold start**.

A cold start typically adds **100 milliseconds to several seconds** of delay to the first request after a period of inactivity.

Old Computer Analogy:

Remember turning on an old Windows computer? It takes a minute before you can do anything useful — loading the operating system, starting background services, initializing hardware. But once it is on and warm, everything works instantly.

A cold start is exactly that first boot delay. If the computer was already on (a “warm start”), everything is instant. But if it was off, you wait.

In serverless: the first request after idle time = cold start (slow). Subsequent requests = warm starts (fast).

Situation	What Happens	Speed
First request after being idle	Container must be created and function loaded	Slower (cold start)
Subsequent requests quickly after	Container is already running (warm)	Fast (warm start)
Function not used for a long time	Container is shut down again	Next request is cold again

Cloud providers are constantly working on reducing cold start times. Techniques like “provisioned concurrency” (keeping a few containers always warm) exist but add cost.

2.8 Limitations of Serverless and FaaS

Serverless is powerful but not suitable for every type of task.

Things Serverless is NOT Good For:

- ✗ **Long-running tasks:** AWS Lambda has a maximum execution time of 15 minutes. Google Cloud Functions have a 9-minute limit. Any task that takes longer simply cannot run as a serverless function.
- ✗ **Local file storage:** A function runs on a randomly assigned machine. Even if you write a file to that machine's disk, the next invocation might run on a completely different machine and that file will be gone. Local storage in serverless is temporary and unreliable.
- ✗ **Consistent performance:** Cold starts introduce unpredictable latency. For applications where every millisecond matters at all times, serverless can be inconsistent.
- ✗ **Complex multi-step workflows:** Tasks that require maintaining state across multiple steps are hard to implement without additional services.

Things Serverless is GREAT For:

- ✓ Sending emails or notifications triggered by an event
- ✓ Processing uploaded files (resize images, convert formats)
- ✓ Handling API requests (backend of a mobile app)
- ✓ Running scheduled jobs (daily data cleanup at midnight)
- ✓ Processing messages from a queue
- ✓ Responding to webhooks from other services

3 MCQ Solutions — Dew Computing & Serverless

How this section works:

Each question gets a full explanation built from the ground up — just like the concept sections above. No jargon is assumed. The **Exam Answer** box at the end of each question gives you the precise, exam-ready response.

3.1 Q1 Dew Computing vs Edge Computing — The Key Difference

The Question: What is the most distinctive characteristic that separates dew computing from edge computing?

Both dew computing and edge computing try to reduce dependence on the cloud by doing some work closer to the user. But they do this in fundamentally different ways, and understanding the exact difference is what this question tests.

Edge computing processes data quickly on nearby devices or servers, but it typically still requires a *network connection* to those nearby resources. A smart factory sensor connects to an edge server in the factory building. A smartphone uses a nearby edge node. The processing is faster because it is local, but the device is still network-dependent.

Dew computing goes a step further. The device itself — your phone, laptop, or tablet — becomes fully self-sufficient. It does not need to connect to anything at all to function. All processing and storage happen on the device. The cloud is used only for optional synchronization when convenient.

The Best Analogy to Remember This:

Edge computing is like having a very fast local library branch near your home. You still go there (make a network request), but it is faster than going to the central library in the city.

Dew computing is like having all the books you need right in your own home. You do not need to go anywhere at all. You can read, work, and study completely independently. When you feel like returning borrowed books (syncing), you can — but only when convenient.

Correct Answer: Dew computing devices can operate **completely offline** and sync with the cloud when connectivity becomes available. This is the distinctive feature that edge computing does not provide — edge still requires network access to function.

3.2 Q2 Best Use Case for Dew Computing

The Question: Which environment benefits most from dew computing?

Dew computing was specifically designed for situations where internet connectivity is unreliable, intermittent, expensive, or completely absent. Its offline-first design philosophy makes it particularly powerful in exactly these circumstances.

Think about the contrast between environments:

A tech company office in a city has high-speed fiber internet, redundant network connections, and cloud apps work perfectly all the time. Dew computing adds little benefit here.

Now consider a village school, a health clinic in a mountain town, a fishing boat far from shore, or a train passing through a tunnel. In all these cases, internet is unreliable or unavailable. Standard cloud apps break down. Dew computing apps keep working.

The Village Education Example Revisited:

A government initiative gives tablets to students in rural schools. The school has solar power but internet only arrives via satellite and is slow and intermittent.

With dew computing apps: students can watch educational videos stored locally, take quizzes on their tablets, get immediate feedback from the app's local AI, and have their progress saved. When the teacher connects the tablets to the satellite internet window at the end of the day, all student data syncs to the central education department's cloud server.

The students learned effectively all day long without a single internet connection during lessons.

Correct Answer: Environments with **unstable or no internet connectivity** — such as rural education, remote healthcare, or offline-first mobile applications — benefit most from dew computing because it allows full productivity regardless of network availability.

3.3 Q3 Who Resolves Data Conflicts After Offline Work?

The Question: When two users edit the same document offline and then sync, which layer is responsible for resolving the conflict?

This question is about understanding the role of each layer when dew computing's biggest challenge — data consistency — occurs in practice.

When offline, your device has no idea what another person's device is doing. You might both be editing the same shared document or the same database record. Both create valid local versions. Neither is wrong from the local perspective. The conflict only becomes apparent when both devices try to sync.

The Group Project Scenario:

You and two friends are collaborating on a group project document. At 9am, all three of you download the document — it has 8 pages. Now you go to different locations with no internet.

You add two new chapters (now 10 pages). Friend A changes the introduction (still 8 pages, different content). Friend B deletes a chapter (now 7 pages, different content).

At 6pm, all three of you reconnect. Three different versions of the document now try to sync to the cloud.

Who decides what the final document should look like? The cloud layer.

The cloud layer has global visibility — it can see all three versions, compare timestamps of changes, analyze which parts were changed versus which were unchanged, and apply conflict resolution algorithms. Common strategies include:

Last-write-wins (the most recent sync overrides older ones), three-way merge (compare all three versions and intelligently combine changes that do not conflict), or flagging conflicts for the user to resolve manually (like Google Docs’ “resolve conflict” prompt).

Correct Answer: The **cloud layer** is responsible for conflict resolution. It has the complete view of all versions, timestamps, and change histories needed to intelligently merge or resolve conflicts that arise when multiple offline devices sync simultaneously.

3.4 Q4 The Core Trade-Off in Dew Computing

The Question: What is the fundamental trade-off introduced by dew computing?

Every design decision in engineering involves a trade-off — you gain something in one dimension but give something up in another. Dew computing’s trade-off is one of the most elegant examples of this principle.

The gain is enormous and clear: you achieve **local autonomy** — the ability to work freely without depending on any external network or server. This is the entire value proposition of dew computing.

The cost is equally real: **data consistency becomes difficult**. When your device and another device both operate independently for hours or days, their views of shared data can diverge significantly. Bringing them back into agreement requires complex synchronization logic, and sometimes requires human judgment.

The Fundamental Trade-Off in One Sentence:

You gain: The freedom to work offline without interruption, completely independent of any network.

You give up: The guarantee that your data is always consistent with what other users and the cloud have — because diverging offline versions create conflicts that are complex to resolve.

This is often described as the trade-off between **availability** (the system is always available to use) and **consistency** (the data is always perfectly up-to-date everywhere). In dew computing, availability is prioritized over consistency.

Correct Answer: The fundamental trade-off is between **local freedom (offline availability)** and **data consistency**. Dew computing prioritizes keeping the device usable at all times, at the cost of introducing potential data conflicts and synchronization complexity when devices reconnect.

3.5 Q5 The Cold Start Problem in Serverless Computing

The Question: What is the primary performance issue in serverless computing?

Serverless functions are not always running. When no requests are coming in, the cloud provider shuts down the function's container to save resources. When a new request arrives after a period of inactivity, the provider must start everything up again from scratch before the function can run. This startup time is the **cold start latency**.

Let us understand exactly what happens during a cold start to appreciate why it causes a delay:

What Happens During a Cold Start:

1. The cloud provider receives the trigger (e.g., an API request arrives).
2. The system must find or create a container (a small virtual environment) to run your function.
3. The container boots up — the operating system and runtime environment (e.g., Python, Node.js) must start.
4. Your function code is loaded into the container's memory.
5. Any connections your function needs (database connections, etc.) must be initialized.
6. Finally — after all that setup — your actual code runs.

All of steps 1–5 are overhead. They add between **100ms and several seconds** to the response time.

The Rebooting Computer Analogy:

You press the power button on a computer that was off. Nothing happens immediately. You wait for the BIOS to load, the operating system to boot, the desktop to appear, and your applications to initialize. Only then can you start

doing work.

Now imagine you were already using the computer (it was warm and running). You switch between apps instantly. No delay.

Cold start = turning on a computer from zero. Warm start = switching between already-open apps.

Correct Answer: Cold start latency — the delay that occurs when a serverless function has been idle and its container must be initialized from scratch before the function can execute, adding unpredictable overhead to the first request after a period of inactivity.

3.6 Q6 Why Serverless Functions Must Be Stateless

The Question: Why is stateless design essential for serverless functions?

Stateless means each function execution is completely isolated and self-contained — it has no memory of previous executions and does not store any information that future executions will rely on.

This might sound like a limitation, but it is actually what makes serverless functions incredibly powerful and scalable. Let us think about why.

Imagine if functions were stateful — they remembered things between runs. Then every request from User A would need to go to the specific function instance that remembers User A's data. You could not freely distribute requests across many instances. You could not restart instances when they crash. You would have complex coordination between instances. The system would become fragile and difficult to scale.

The Teller Window Analogy:

Imagine a bank with 20 teller windows. If each customer must always go to the same specific teller (because only that teller “remembers” their account), then:

If Teller 7 is sick, all of Teller 7's customers cannot be served. If a lot of customers arrive at once, you cannot easily add new tellers. The whole system depends on continuity.

Now imagine any teller can serve any customer, because customer account data is always available in the central bank database — not in any teller's memory. Now:

If Teller 7 is sick, customers go to any other teller. High traffic? Open more teller windows instantly. This is stateless design. Each teller (function instance) is interchangeable.

Correct Answer: Stateless design allows serverless functions to **scale infinitely and independently**. Any instance can handle any request without coordination. New instances can be added instantly under load. Failed instances can be replaced without data loss. Stateful functions would create dependencies that prevent this kind of effortless scaling.

3.7 Q7 Why FaaS is Not Suitable for Long-Running Tasks

The Question: What makes FaaS inappropriate for long-running computational jobs?

FaaS platforms are built with a fundamentally different purpose in mind compared to traditional computing. They are optimized for **fast, event-driven, short tasks** — things that start quickly, do a specific job, and finish. They are explicitly not designed for tasks that run for hours.

This is enforced through strict **execution time limits**:

Platform	Maximum Time	Execution	If Exceeded
AWS Lambda	15 minutes		Function is forcibly terminated
Google Cloud Functions	9 minutes		Function is forcibly terminated
Azure Functions	5 minutes (default)		Function is forcibly terminated

These limits exist by design, not by accident. They enforce the short-task philosophy and allow the platform to efficiently reclaim resources.

The Sprint vs Marathon Analogy:

A sprint runner is fast over 100 meters but cannot maintain that pace for 42 kilometers. A marathon runner is designed for endurance.

FaaS is a sprint system. It is incredibly fast and efficient for short bursts. But if you ask a sprinter to run a marathon, they will collapse before the finish line. Similarly, if you try to run a 3-hour data simulation as a Lambda function, it will be forcibly shut down at the 15-minute mark.

For long tasks, you need a marathon runner — a traditional server or a dedicated compute job service like AWS Batch or a long-running container on Kubernetes.

Correct Answer: FaaS platforms enforce strict **execution time limits** (e.g., AWS Lambda: 15 minutes maximum). Any function running longer is forcibly terminated. Long-running jobs like machine learning training, large data transformations, or continuous simulations are fundamentally incompatible with this model.

3.8 Q8 Why Local File Storage is Unreliable in FaaS

The Question: Why cannot serverless functions reliably use local file storage?

This is a direct consequence of the stateless and ephemeral nature of serverless functions. When a function runs, it is placed in a container on some machine in the cloud. That function can write a file to that machine's local disk.

But here is the critical insight: **the next time the function is triggered, it may run on a completely different machine.** The previous machine's disk is not accessible from this new machine. The file you wrote is gone — from your function's perspective.

Even worse, if the function is scaled to run many parallel copies simultaneously, each copy runs on a different machine with its own separate local storage. There is no shared disk between them.

The Post-it Note Analogy:

Imagine you are working in a large office where your desk changes every day. You leave a Post-it note on today's desk with important information. Tomorrow, you come in and are assigned a completely different desk. The Post-it note from yesterday's desk is not there. The office cleaning staff may have removed it. Even if they did not, you are at a different desk and cannot see it.

Saving a file to a serverless function's local storage is exactly like writing a Post-it note on a temporary desk. You cannot rely on finding it next time.

The Right Solution — External Storage:

- ✓ Store files in cloud object storage (like AWS S3, Google Cloud Storage)
- ✓ Store data in a database (like DynamoDB, Firestore, RDS)
- ✓ Store temporary state in a cache (like Redis)

These are shared, persistent, and accessible by any function instance — regardless of which machine it runs on. The function reads from and writes to these external systems, never trusting its own local disk for anything important.

Correct Answer: Serverless functions are **ephemeral and stateless** — each invocation may run on a different machine. Local disk storage is not shared between invocations or parallel instances and is erased when the container is shut

down. Any important data must be stored in external, persistent storage such as cloud object stores or databases.

3.9 Q9 AWS Lambda — Maximum Execution Time

The Question: What is the maximum execution time limit for a single AWS Lambda function invocation?

This is a factual question about AWS Lambda, the most widely used FaaS platform in the world. AWS Lambda was launched in 2014 and popularized the serverless computing model.

The execution time limit has changed as the service matured:

AWS Lambda Limit	Value
Maximum execution time	15 minutes (900 seconds)
Default memory	128 MB (configurable up to 10 GB)
Maximum payload size	6 MB (synchronous), 256 KB (async)
Default timeout	3 seconds

Practical Implication:

If you are processing a video file that takes 20 minutes to compress, AWS Lambda is the wrong tool. Lambda will run for 15 minutes and then your function is killed — no output, wasted compute cost, and potentially corrupted output files.

The correct approach for tasks over 15 minutes is to use AWS Fargate (containerized task), AWS Batch (managed batch jobs), or break the task into smaller chunks that each fit within the 15-minute limit.

Correct Answer: AWS Lambda has a maximum execution time of **15 minutes (900 seconds)** per invocation. Any function that exceeds this limit is forcibly terminated by the AWS platform.

3.10 Q10 Amazon Redshift — What Is It and Who Provides It?

The Question: Amazon Redshift is a product of which cloud provider, and what category does it belong to?

Amazon Redshift is a *data warehouse* service provided by **Amazon Web Services (AWS)**.

A data warehouse is a special type of database designed not for running an application (like storing users or orders), but for **analyzing huge amounts of data**. It stores data in a format optimized for running complex analytical queries — things like “What was the total revenue per country per quarter over the past three years?”

What Redshift Is Used For:

Imagine a company like Amazon itself, with billions of transactions, millions of products, and hundreds of millions of customers. Every day, analysts want to run questions like:

“Which product categories grew the most in Southeast Asia this quarter?”

“At what time of day do users from tier-2 cities make the most purchases?”

“How did the monsoon season affect food delivery orders last year?”

These are not simple lookups. They involve scanning terabytes or petabytes of historical data and performing complex aggregations. A normal database would take hours or crash. Redshift is purpose-built for this — it can answer these questions in seconds using parallel processing across thousands of CPU cores.

Property	Detail
Provider	Amazon Web Services (AWS)
Category	Data Warehouse (OLAP — Online Analytical Processing)
Storage Format	Columnar (reads only the columns needed for a query — very fast)
Scale	Petabytes of data
Use Case	Business intelligence, historical analysis, reporting dashboards
NOT For	Real-time transactions, app backends, low-latency single-record lookups

Correct Answer: Amazon Redshift is a **fully managed, petabyte-scale data warehouse** provided by **Amazon Web Services (AWS)**. It is used for large-scale analytical queries on historical business data, not for real-time transactional application backends.

4 Master Quick-Reference Sheet

4.1 Dew Computing — Key Facts

Concept	One-Line Explanation
What is Dew?	Work locally on your device; sync to cloud when internet is available
Why “Dew”?	Dew is the water layer closest to the ground — your own device is the closest layer
Key feature	Fully offline capable — no internet required to function
Best use case	Rural/remote areas with unreliable internet; offline-first apps
How it syncs	Automatically when connectivity is detected — no manual action needed
vs Edge	Edge still needs a network; Dew works completely without one
Trade-off	Offline freedom gained; data consistency problems introduced
Who resolves conflicts	The cloud layer, using timestamps, merge algorithms, or user prompts

4.2 Serverless Computing — Key Facts

Concept	One-Line Explanation
What is serverless?	You write code; cloud runs it; you pay only when it runs
Are there servers?	Yes, but you never see or manage them
What triggers a function?	Events: user action, file upload, timer, API request, message queue
Stateless	Each run is independent; no memory of previous runs
Why stateless?	Enables infinite scaling with no conflicts between instances
Cold start	Delay when function wakes up after being idle (container must restart)
FaaS time limit	AWS Lambda: 15 min; Google: 9 min; Azure: 5 min (default)
Local storage?	Unreliable — next run may be on a different machine; use external storage
What is Redshift?	AWS data warehouse for petabyte-scale analytical queries

4.3 The Computing Hierarchy — All Layers Together

Layer	Location	Internet Needed	Primary Role
Cloud	Remote data center	Always	Heavy compute, storage, analytics
Fog	Nearby local server	Mostly	Filter data, reduce cloud load
Edge	On/near device	Sometimes	Real-time decisions
Dew	Your own device	Never (optional)	Offline-first, local autonomy

4.4 Five Golden Rules to Remember

Rule 1: Dew computing's defining feature is **complete offline operation**. Edge computing still needs a network. This one distinction separates them.

Rule 2: The trade-off in dew computing is always **availability vs consistency**. You gain offline freedom; you create data conflict challenges.

Rule 3: The **cloud layer** resolves sync conflicts in dew computing — the device handles local work, the cloud handles reconciliation.

Rule 4: Cold start latency is the **primary performance limitation** of serverless. The first request after idle time is always slower.

Rule 5: In FaaS, **never trust local disk storage** and **never write long-running tasks**. Use external storage (S3, databases) and stay within time limits (Lambda: 15 min max).